Smart Display · Group 5א · IoT project 236333, Technion

HW Unit Tests Report

Final submission: every hardware part and service the display depends on, tested on its own

Shada Nujeidat, Yuval Carmelli, Liat Rokach · 12 October 2026

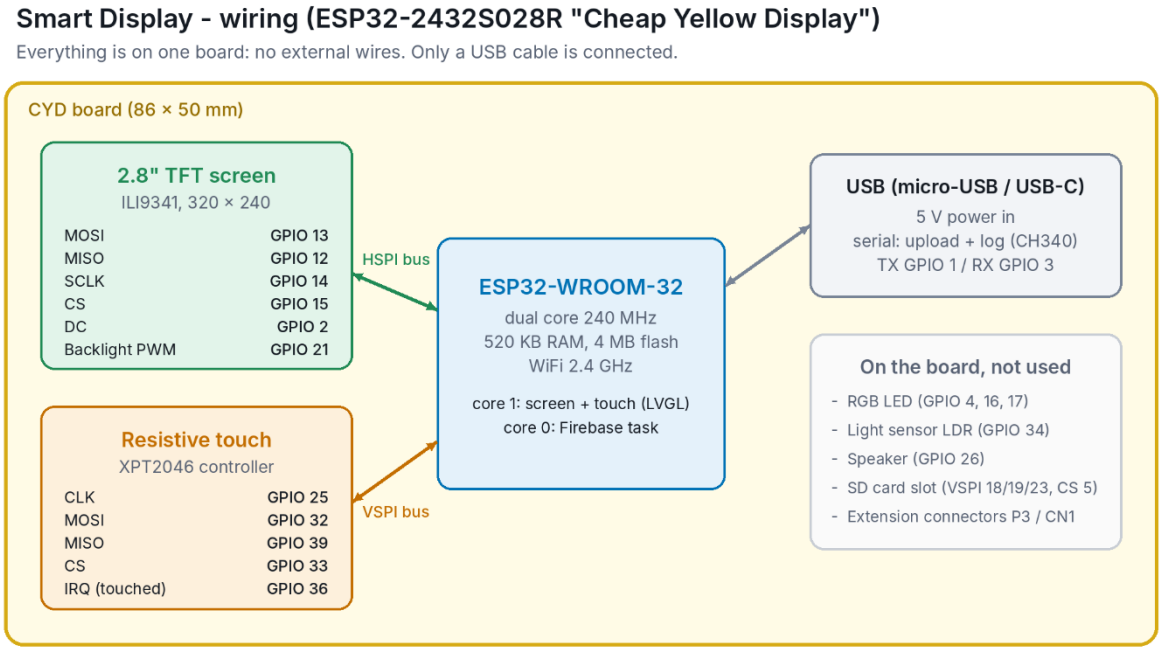
1. What we test and why

The Smart Display runs on one board, the **ESP32-2432S028R** ("Cheap Yellow Display", CYD): an ESP32 with a 2.8" 320×240 touchscreen. Everything else (weather, time, calendar, voice commands) arrives over WiFi. For each hardware part and each online service we wrote a small stand-alone sketch that tests only that part, outside the full firmware. When something stops working, these tests tell us quickly whether the cause is the hardware, the network, a service, or our own code.

All 9 sketches are in the Git repository under `unit_tests/`, each with its own folder. Each prints its measurements to the Serial Monitor and ends with one line, `RESULT: PASS` or `RESULT: FAIL`. They use the same pins, settings and conversions as the firmware, so a passing test means the firmware's configuration is right too.

Hardware under test

Part	Chip / interface	Pins (GPIO)	Used by the firmware for
Screen	ILI9341 TFT, SPI (HSPI) at 55 MHz	MOSI 13, MISO 12, SCK 14, CS 15, DC 2	all screens (LVGL)
Touch	XPT2046 resistive, SPI (VSPI)	CLK 25, MOSI 32, MISO 39, CS 33, IRQ 36	every button, slider, list tap
Backlight	LED driver, PWM 5 kHz, 8 bit	21	brightness, dim / screen off



Pins as configured in main/build_opt.h (screen) and main/src/touch/display_ui.h (touch). Screen and touch use separate SPI buses, so they never block each other.

Part	Chip / interface	Pins (GPIO)	Used by the firmware for
Flash (NVS)	ESP32 internal flash, Preferences	-	settings, shopping / to-do lists
WiFi	ESP32 radio, 2.4 GHz	-	weather, time, calendar, Firebase, setup portal
Not used	RGB LED, light sensor (LDR), speaker, SD card slot	4/16/17, 34, 26, 5/18/19/23	not part of the project, not tested

Figure 1. CYD internal wiring (all parts are on the board, no external wiring). Also in the repo: assets/wiring_diagram.png

Test setup

- Arduino IDE 2, board package **esp32 by Espressif 3.3.12**, board **ESP32 Dev Module**, Partition Scheme **Huge APP**.
- Libraries (same pinned versions as the firmware): TFT_eSPI 2.5.43, XPT2046_Touchscreen 1.4.0, ArduinoJson 7.4.3. No extra libraries.
- Serial Monitor at 115200 baud; press RST/EN after opening it to see the output from the start.
- Network tests use the network saved by the display's setup portal (SmartDisplay-Setup), or one typed into `TEST_WIFI_SSID`. Run them on the network the display will really use (Technion WiFi and a phone hotspot).
- The Firebase and Calendar tests need a copy of `main/secrets.h` in their folder. These copies are in `.gitignore` and are never uploaded.

2. Summary of results

Run the tests in this order: hardware first, then network, then the services. If a lower test fails, the ones above it are the first suspects.

#	Test (sketch)	Pass criterion	Result	Measured
1	Screen `test_display`	fill ≤ 60 ms; border, corners, colors, text correct	**PASS**	fill 31 ms; visual check correct
2	Touch `test_touch`	all 5 taps within 20 px	**PASS (2nd run)**	worst error 17 px
3	Backlight `test_backlight`	every step brighter, 0 % dark, no flicker	**PASS**	visual check correct
4	Flash storage `test_storage`	values match, counter +1 after RST	**PASS**	2 KB write 11 ms
5	WiFi `test_wifi`	connect + reconnect ≤ 15 s, 4 names resolve	**PASS (2nd run)**	connect 1201 ms, RSSI -44 dBm, reconnect 1101 ms
6	Clock (NTP) `test_ntp`	zone rule right, sync ≤ 20 s, drift ≤ 1 s	**PASS**	sync 19.6 s, no drift in 60 s
7	HTTPS / TLS `test_https_weather`	200 with 5 cities, dead server < 10 s	**PASS**	request 1387 ms, TLS ~49 KB, dead server 8.0 s
8	Firebase `test_firebase`	sign-in, write/read match, delete	**PASS**	sign-in 1056 ms, average read 962 ms

#	Test (sketch)	Pass criterion	Result	Measured
9	Google Calendar `test_calendar`	200, start times present, Hebrew intact	**PASS**	request 1209 ms, 5 events

Test Date: 12 October 2026 · Board: ESP32-2432S028R (CYD) · Network: home WiFi, 2.4 GHz · Result: all 9 tests passed

3. Test procedures

1. Screen: test_display

Part	ILI9341 2.8" TFT, 320×240, HSPI (pins 13/12/14/15/2)
Goal	The screen is wired and configured correctly: fast enough, nothing cut off or mirrored, correct colors.
Procedure	1. Upload `unit_tests/test_display` (its `build_opt.h` holds the same screen settings as the firmware). 2. The screen fills red, green, blue, white, black; each fill is timed and printed. 3. Then it draws a 1 px white border, a colored square in each corner, a center cross and text in 3 sizes. 4. Check the screen against the list printed in the Serial Monitor.
Pass criterion	Each fill ≤ 60 ms (expected ≈ 22 ms at 55 MHz SPI). Border touches all 4 edges. Red square top-left, green top-right, blue bottom-left, yellow bottom-right. Text sharp.
If it fails	White screen: screen configuration not loaded (sketch opened from the wrong folder). Red top-right: mirrored, wrong rotation. Wrong colors: inversion setting, or a board with an ST7789 panel.
Result	**PASS**: Full-screen fill 31 ms (limit 60 ms). Border on all 4 edges, corner colors correct, text sharp.

2. Touch: test_touch

Part	XPT2046 resistive touch, VSPI (CLK 25, MOSI 32, MISO 39, CS 33, IRQ 36)
Goal	The touch controller answers and a tap lands where the user pressed, using the same conversion as the firmware (`map(x, 200, 3700, …)`, `map(y, 240, 3800, …)`).
Procedure	1. Upload `unit_tests/test_touch`. A cross appears at each of 5 targets: 4 corners and center. 2. Tap the center of each cross with a fingernail or stylus (20 s per target). 3. The sketch averages 8 readings, converts them to pixels, prints target, tap, raw values and the error in pixels, and marks the tap green (ok) or red.
Pass criterion	Every tap within 20 px of its target. The firmware's smallest button is 44 px, so a 20 px error still hits the right button.
If it fails	No reaction: wrong touch pins or SPI bus. Large error in one direction: adjust the `map()` limits in `display_ui.h` using the raw values printed by the test (this is our calibration procedure).
Result	**PASS (2nd run)**: Worst error 17 px (limit 20 px); other taps 14, 8, 10, 11 px. 1st run FAIL (88-169 px), see section 4.

3. Backlight: test_backlight

Part	Backlight LED driver, PWM on GPIO 21 (5 kHz, 8 bit)
Goal	Brightness follows the PWM duty cycle used by Settings → Brightness and by power saving (dim / screen off).
Procedure	1. Upload `unit_tests/test_backlight`. The backlight steps through 0 %, 4 % (the firmware's minimum), 10 %, 25 %, 50 %, 75 %, 100 %, 2 s each, then fades up and down.
Pass criterion	Each step visibly brighter than the previous one, 0 % fully dark, no flicker at any level, smooth fade.
If it fails	Always full brightness: GPIO 21 not attached to PWM. Flicker at low levels: PWM frequency too low.
Result	**PASS**: Every step visibly brighter, 0 % fully dark, no flicker (visual check).

4. Flash storage: test_storage

Part	ESP32 flash, NVS partition (Preferences library)
Goal	Settings and the shopping / to-do lists are saved in flash and survive a restart.
Procedure	1. Upload `unit_tests/test_storage`. It writes and reads back an int, a Hebrew string and a 2 KB block (the size of the saved lists), timing the block write. 2. It increases a boot counter on every start. Press RST/EN once: the counter must be one higher. 3. Uses its own namespace ("unittest"), so the display's real settings and lists are not touched.
Pass criterion	All values read back identical, the Hebrew string is unchanged, the counter increases after RST.
If it fails	Read-back differs or counter resets: NVS partition full or corrupted (erase flash and re-upload).
Result	**PASS**: Int, Hebrew string and 2 KB block read back correctly; 2 KB write 11 ms; boot counter kept across restarts (4); 321 free NVS entries.

5. WiFi: test_wifi

Part	ESP32 WiFi radio (2.4 GHz)
Goal	The radio finds networks, connects, resolves the servers the display uses, and reconnects after a drop.
Procedure	1. Upload `unit_tests/test_wifi`. It uses the network saved by the display's setup portal, or the one typed into `TEST_WIFI_SSID`. 2. Prints a scan (name, signal, channel), connect time, RSSI and IP. 3. Resolves api.open-meteo.com, identitytoolkit.googleapis.com, www.googleapis.com, ntp.technion.ac.il. 4. Disconnects on purpose and measures the reconnect.
Pass criterion	Connects within 15 s, all 4 names resolve, reconnects within 15 s.
If it fails	Phone hotspot not found: switch the iPhone hotspot to "Maximize compatibility" (2.4 GHz). DNS fails: network blocks outside traffic (captive portal).
Result	**PASS (2nd run)**: 3 networks found; connected in 1201 ms, RSSI -44 dBm; all 4 names resolved; reconnect 1101 ms. 1st run FAIL (scan found 0 networks), see section 4.

6. Clock: test_ntp

Part	ESP32 RTC + NTP over WiFi
Goal	The clock shows Israel time, including summer time, synced from a server that works on the Technion network.
Procedure	1. Upload `unit_tests/test_ntp`. First, without network, it converts two fixed moments with the firmware's time zone rule: 15 Jan 2026 12:00 UTC must be 14:00, 15 Jul 2026 12:00 UTC must be 15:00. 2. Then it syncs from ntp.technion.ac.il (backups: pool.ntp.org, time.google.com) and prints the local time. 3. After 60 s it compares the clock with millis().
Pass criterion	Both conversions right, synced within 20 s, drift ≤ 1 s per minute.
If it fails	Never syncs on campus: the Technion network blocks outside NTP servers, so the Technion server must stay first.
Result	**PASS**: Winter/summer conversions right; synced in 19.6 s (limit 20 s); clock moved 60 s in 60 s (no drift).

7. Secure requests: test_https_weather

Part	TLS (mbedTLS) + HTTP client, RAM
Goal	The real weather request works over HTTPS, fits in memory, and a dead server cannot freeze the screen.
Procedure	1. Upload `unit_tests/test_https_weather`. It sends the firmware's request: weather for all 5 cities in one call. 2. Prints status, time, size, free memory before and the lowest during the request, and the 5 temperatures. 3. Then it connects to an address that never answers (10.255.255.1) and measures how long it takes to give up.
Pass criterion	HTTP 200 with 5 cities. The dead server gives up in under 10 s (the default would be 120 s).
If it fails	Restart or error -1 during the handshake: not enough free memory (TLS needs about 40 KB).
Result	**PASS**: HTTP 200 in 1387 ms, 3996 bytes, all 5 cities; free memory 224,840 B before, 175,972 B lowest (~49 KB used by TLS); dead server gave up after 8.0 s.

8. Firebase: test_firebase

Part	Firebase Auth + Realtime Database (REST)
Goal	The display's login works and the database path that voice commands use can be written and read fast enough.
Procedure	1. Copy `main/secrets.h` into the test folder (it is in .gitignore). Upload `unit_tests/test_firebase`. 2. Signs in with the display's email/password, writes a random value to `/unit_test`, reads it back 5 times (timed), deletes it. `/command`, `/status` and `/lists` are not touched.
Pass criterion	Sign-in OK, the value read back matches every time, delete OK. Average read well under 1 s (the display checks /command once a second).
If it fails	INVALID_PASSWORD / EMAIL_NOT_FOUND: wrong secrets. 401 on write: database rules. Slow reads: weak WiFi signal (see test 5).
Result	**PASS**: Sign-in 1056 ms; write 200; 5 reads matched, average 962 ms, worst 1118 ms; delete 200. Earlier FAIL was a bug in the test, see section 4.

9. Google Calendar: test_calendar

Part	Google Calendar API v3 (public calendar + API key)
Goal	The calendar ID and API key work and the reply contains what the calendar screen needs, including Hebrew titles.
Procedure	1. Copy `main/secrets.h` into the test folder. Upload `unit_tests/test_calendar`. 2. Requests the first 5 events since 1 Jan 2026 and prints their start time and title.
Pass criterion	HTTP 200, every event has a start time, Hebrew titles print as Hebrew (not ???).
If it fails	404: wrong calendar ID or the calendar is not public. 400/403: wrong key, or Google Calendar API not enabled in the Cloud project.
Result	**PASS**: HTTP 200 in 1209 ms; 5 events with start times; Hebrew titles printed correctly.

4. What we learned from the test run

Some tests did not pass on the first try. We kept these runs in the report because each one taught us something about the hardware, the network or the tests themselves.

Test	What happened	Cause and what we did
WiFi	1st run: the scan found 0 networks and the board did not connect. 2nd run right after: everything passed	The radio was not fully started when the scan ran. The firmware is not affected: it retries the saved network every 15 s

Test	What happened	Cause and what we did
Touch	1st run: taps landed 88-169 px from the targets, in random directions. 2nd run: worst 17 px	Random errors (not a constant offset) come from pressing too lightly or sliding the finger, which resistive screens read badly. A firm press with a fingernail gives accurate results. The calibration itself is correct; the raw values show the X range could be widened slightly
Clock (NTP)	Synced in 19.6 s, just under the 20 s limit	At home the Technion time server answers slowly, so the board waits before using the backup servers. On campus it is the fast path. The display only shows "Getting the time..." a bit longer after starting
Firebase	1st run: "value WRONG" although sign-in, write, read and delete all worked	A bug in the test: the random value could be larger than the type used to read it back. Fixed by keeping the value below 1,000,000 and printing any mismatch; the rerun passed
Google Calendar	1st run: HTTP -1 after 12 s (no connection)	The test sent the request the moment WiFi connected, before the connection was ready. The test now waits until the server name resolves and retries once; the rerun passed. The firmware only asks for the calendar after the clock is synced, so it is not affected
Flash storage	No output in the Serial Monitor	The test finished in under a second, before the monitor was open. It now waits 3 s and repeats the result every 5 s
Screen	Full-screen fill 31 ms instead of the estimated 22 ms	The estimate counted only the pixel data; the board also spends time on commands and the SPI driver. 31 ms is well under the 60 ms limit

5. Problems found while bringing up the hardware

These came up while we first ran examples and the early firmware on the board. Each is now handled in the firmware, and the test in the last column would catch it again.

Problem	Cause	Fix	Caught by
Arduino Blink example: nothing blinks	The CYD has no LED on LED_BUILTIN; its RGB LED is on GPIO 4/16/17 and is active-LOW	Used the RGB pins with inverted logic (LED not used in the final project)	-
Screen white, or colors wrong	TFT_eSPI needs the CYD's pins and driver; this panel also needs color inversion	Screen configuration in `build_opt.h` (ILI9341_2 driver, HSPI pins, inversion on), no edits to the library	test_display
Touch position off from the finger	Raw XPT2046 values must be converted to pixels; the limits differ slightly per board	Conversion `map(x, 200, 3700)`, `map(y, 240, 3800)`, checked against 5 targets; buttons ≥ 44 px	test_touch
A tap registered twice	Resistive touch bounces when pressed lightly	LVGL acts on one complete press-and-release and ignores presses that turn into a scroll; ticked list items can be undone within 2 s	test_touch (manual)
Could not reliably join an iPhone hotspot	The ESP32 only supports 2.4 GHz; newer iPhones use 5 GHz unless "Maximize Compatibility" is on. The example also did not set station mode	`WiFi.mode(WIFI_STA)`; hotspot set to "Maximize compatibility"	test_wifi
Clock never set on the Technion network	Outside NTP servers are blocked on campus	ntp.technion.ac.il first, pool.ntp.org and time.google.com as backups	test_ntp
Screen froze up to 2 min when WiFi dropped mid-request	Default TLS handshake timeout is 120 s	Handshake timeout 8 s (15 s for Firebase)	test_https_weather
"Sketch too big" after adding LVGL	The default app partition is 1.3 MB	Partition scheme Huge APP (3 MB); firmware is now 1.61 MB (51 %)	build
Hebrew list items and events showed ???	The built-in fonts have no Hebrew letters	Hebrew fonts with right-to-left support (LVGL BiDi)	test_calendar, test_storage

Problem	Cause	Fix	Caught by
Light mode looked grey on the panel	The light palette used light-grey backgrounds, which look clearly grey on this panel	Light mode uses pure white backgrounds with outlined rows	test_display (visual)

6. Build check

All 9 test sketches and the full firmware compile and link for ESP32 Dev Module with esp32 core 3.3.12 and the pinned library versions. The full firmware uses 1,606,735 bytes of program space (51 % of the 3 MB Huge APP partition) and 72,772 bytes of RAM for global variables (22 %), leaving room for the secure connections that are created at run time.

7. Where to find it

- Test sketches: Git repository, folder `unit_tests/` (one folder per test).
- How to run them and a results table: `unit_tests/README.md`.
- The pins and settings the tests share with the firmware: `main/build_opt.h`, `Documentation/pinout_wiring.md`, `Documentation/parameters.md`.